Advanced IO

System Input/Output

Stefano Quer
Dipartimento di Automatica e Informatica
Politecnico di Torino

License Information

This work is licensed under the license

CC BY-NC-ND 4.0

Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

BY: Credit must be given to you, the creator.

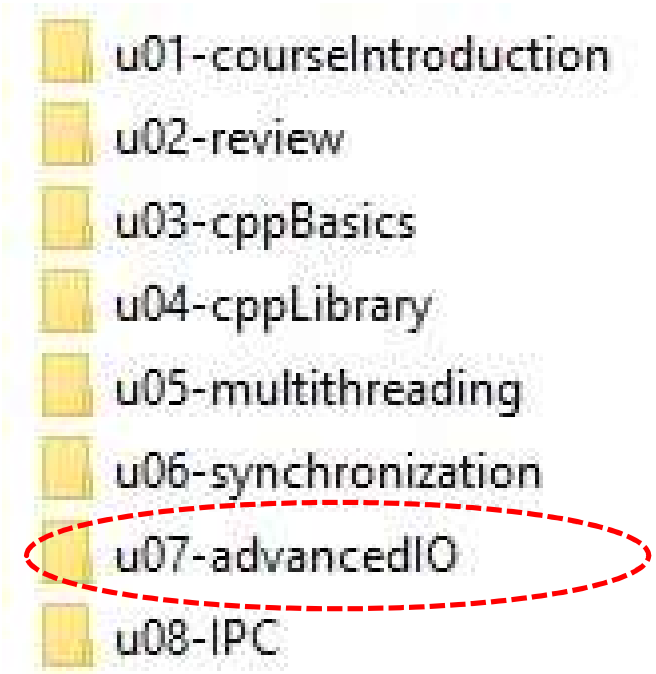
NC: Only noncommercial use of your work is permitted.
Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

ND: No derivatives or adaptations of your work are permitted.

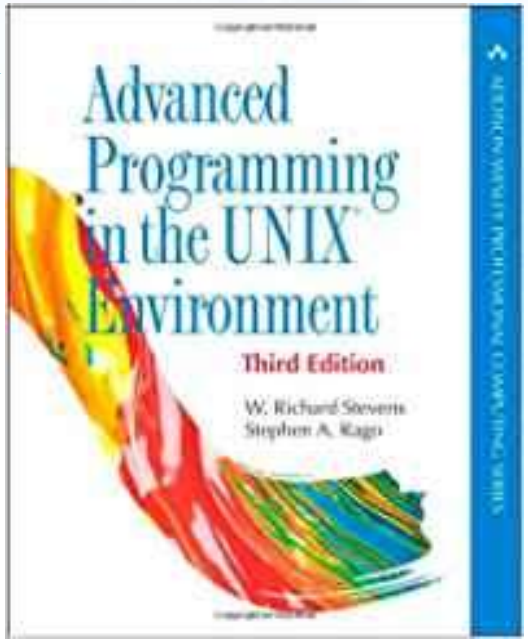
To view a copy of the license, visit:
https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1

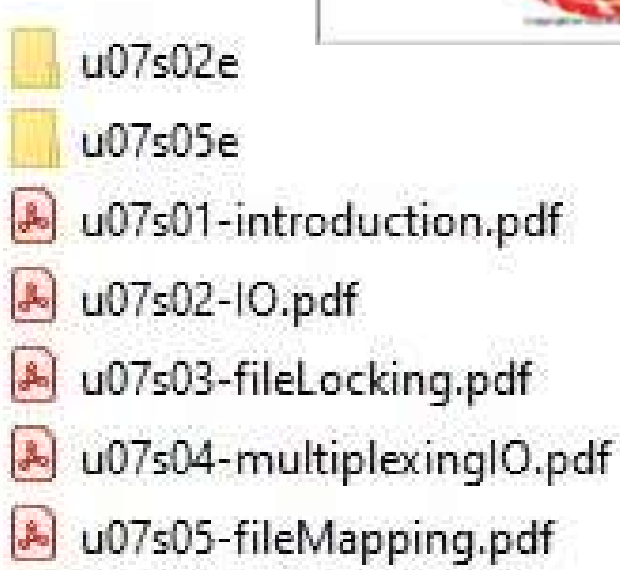
Premises

- Where are we?



Main reference





System calls

- In modern OSs hardware-related services are performed using system calls
  - Access a hard disk drive or a device, create and execute a new process, etc.
- System calls are how a program requests a service from the OS
  - They provide an essential interface between a process and the operating system
  - They are well-defined and limited in number
    - Linux and OpenBSD about 300
    - NetBSD and FreeBSD about 500
    - Windows about 2000

For more details, see the material of the OS course and Prof. Cabodi's part

System calls

- System calls are similar to functions but
  - Require the intervention of the kernel, i.e., a transfer of control from the user space to the kernel space
  - Need some sort of architecture-specific features like a software interrupt or a trap
  - They are by nature slow (as they require the intervention of the kernel)
    - They are usually divided into **slow** and **all the others** system calls

IO system calls

- Slow system calls are the ones that can block forever
  - Many are related to IO operations
    - **Opens** can block until some condition occurs on certain files
      - Until the unit is attached to the system (e.g., a file)
      - Until another process has the unit open for reading (e.g., a FIFO)
    - **Reads** can block if the data is not present
      - Read from stdin or a pipe
    - **Writes** can block if they cannot be accepted
      - A write to a full disk or a full pipe a write on a **locked** record in a file

See section u07s03

Synchronous system calls

- Moreover, standard IO system calls are usually synchronous
  - The task waits until the operation completes
  - Synchronous operations are inherently **slow** compared to other processing
  - Delays may be caused by
    - Hardware devices, e.g., track and sector, seek time on random access, etc.
    - Data transfer between a physical device and the system memory, etc.
    - Network transfer between file servers and computing devices, etc.

IO Efficiency

- In many applications, IO is fundamental and being inefficient implies enourmous time penalty
- The standard UNIX-POSIX introduces numerous advanced IO functionalities such as
  - Non-blocking and asynchronous IO
  - File locking
  - Multiplexing IO
  - Memory-mapped IO